DS5899: Special Topics in Data Science

Leveraging NLP in Asset Management

Week 10: RL

Instructor: Che Guan, Ph.D.

The slides are based on Chapters 3, 4, and 8 of:
Shankar, A. (2024). *Reinforcement Learning for Large Language Models: A Complete Guide from Foundations to Frontiers*

Part 1:

RL Framework

Overview

Every RL system has 5 key pieces. Think of it like a video game:

1.

Agent
(The Player)
The AI making decisions (e.g., the language model).

2.

Environment
(The Game World)
Everything the AI interacts with (e.g., the user, the conversation)

3.

State
Screen/Situation)
What's happening right now (e.g., prompt + text so far)

4.

Action
(Controller Input)
What the AI can do (e.g., choosing the next word)

5.

Reward
(Score/Points)
Feedback for good or bad moves (e.g., how helpful was the response?).

The loop:

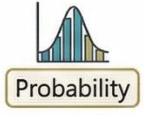
The *Policy*

The **policy** is the agent's decision-making strategy.

It answers: "Given what's happened so far, what should I do next?"

For Language Models:

- **Input:** The prompt and everything written so far



- **Output:** Probability distribution over all possible next words

- **Example:** Given "The cat sat on the", what word comes next?

The Policy Function

General Definition

$\pi(a \mid s)$

Probability of choosing action a when in state s

Language Models Specifically

$\pi_\theta(y_t \mid x, y_{1:t-1})$

Chance of picking word y_t given prompt x and previous words

Symbol Translation:

Symbol	Meaning	Real-World Analogy (Intuition)
π (pi)	= Policy	The brain's decision-making process
θ (theta)	= Parameters	The brain's knowledge (billions of numbers in the model)
a	= Action	The choice to make (which word to write)
s	= State	Current situation (everything so far)
y_t	= Token at time t	The t-th word in the response
x	= Prompt	The user's question/input
$y_{1:t-1}$	= Previous tokens	All words written before word t
$\mid$	= "Given that"	"Knowing..." or "Based on..."

Why condition on previous words with |?

Because language has *context dependence*:

Each choice is formed by what came before it!

Breaking It Down Step-by-Step

1. Understanding the Policy in Action

State `$s$`: "The cat sat on the"

Question: What word should come next?

Policy Output:

Possible Word	Formal	Probability	Percentage
"mat"	$\pi_\theta(\text{mat} \mid s)$	0.35	35%
"floor"	$\pi_\theta(\text{floor} \mid s)$	0.25	25%
"couch"	$\pi_\theta(\text{couch} \mid s)$	0.20	20%
"table"	$\pi_\theta(\text{table} \mid s)$	0.10	10%
"roof"	$\pi_\theta(\text{roof} \mid s)$	0.05	5%
Others	$\pi_\theta(\text{other} \mid s)$	0.05	5%
Total		**1.00**	**100%**

2. How Does the Model Choose?

Weighted Dice: The model uses probabilities for a balanced approach.

Most of the time (35%): picks "mat".

Sometimes (25%): picks "floor".

Rarely (5%): picks "roof".

This randomness is actually good—it helps the model explore different possibilities!

3. What Happens During RL Training?

Part A: Scenario 1: Positive Feedback

"mat" gets high reward **(+8)**

mat` probability **increases** from 0.35 to 0.45

Response "mat" is more likely!

Part B: Scenario 2: Negative Feedback

"roof" gets low reward **(-2)**

roof` probability **decreases** from 0.05 to 0.02

Response "roof" is less likely!

The Key Idea:

Training adjusts these probabilities based on what leads to good outcomes!

Good outcome → Increase probability

Bad outcome → Decrease probability

Over millions of examples, the model learns an excellent policy!

The Value Function

The value function estimates: “How good is my current situation?”.
It’s like looking at a chess board mid-game and thinking:

CHESS ANALOGY (Current state)

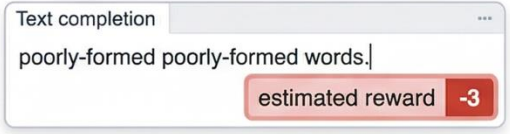
- “I’m in a strong position, likely to win!” (High Value)

- “Uh oh, I’m in trouble…” (Low Value)

LANGUAGE MODELS (Generating a response)

- Halfway through writing a response
- Value function estimates: “This is going well, probably get +8 reward at the end”

- Or: “This is going poorly, probably get -3 reward”



Understanding Reinforcement Learning: The Value Function

Formal Math	What It Really Means
$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^{t} r_t \mid s_0 = s\right]$	Expected total future reward starting from state s
Breaking down each piece and WHY we use it:	
$V^{\pi}(s)$	Value of state s - how good is this situation?
$\mathbb{E}_{\pi}[\cdot]$ 	Expected value - average over many possible futures (we use expectation because future is uncertain)
$\sum_{t=0}^{\infty}$ 	Sum all future rewards from now ($t = 0$) to end (we care about total, not just immediate reward)
γ^{t} 	Discount factor raised to power t (we use this because immediate rewards matter more than distant ones)
r_t	Reward at time step t (the feedback we get)
$\mid$	Condition symbol - given that...
$s_0 = s$	Starting from this specific state s

Deep Dive: Why the discount factor γ (typically 0.99)?

Makes rewards far in the future count slightly less. Why? (1) More uncertainty about distant future, (2) Ensures series converges to finite value, (3) Like money - $100 today worth more than $100 in 10 years. For language models, we care most about the final response quality.

Breaking It Down Step-by-Step

Scenario: You're writing a math solution, currently halfway done.
Current state s: "To solve this, I'll first..."
Possible futures:

✅ **Path A (60% likely):** Complete answer correctly
- Step 1: Write next sentence → reward $r = 0$
- Step 2: Write another sentence → = 0 (neutral)
- Step 3: Finish correct answer → +10 (great!)
- **Total:** 0 + 0 + 10 = 10

B **Path B (30% likely):** Make error, then correct it
- Step 1: Make mistake → −2 (penalty)
- Step 2: Realize and backtrack → reward r_2 = −1 (small penalty)
- Step 3: Fix and complete → r_3 = +8 (good!)
- **Total:** −2 + (−1) + 8 = 5

C **Path C (10% likely):** Give up or wrong answer
- Step 1: Incomplete response → reward r_1 −5 (bad!)
- **Total:** −5

Computing the value $V^{\pi}(s)$:

$$V^{\pi}(s) = \mathcal{P}(\text{Path A}) \times \text{Reward}_A + \mathcal{P}(\text{Path B}) \times \text{Reward}_B + \mathcal{P}(\text{Path C}) \times \text{Reward}_C$$
$$= 0.6 \times 10 + 0.3 \times 5 + 0.1 \times (-5)$$
$$= 6 + 1.5 - 0.5$$
$$= 7$$

Value Analysis for State (s)

Value = 7. This is a high-value state, indicating that on average, continuing with our current policy is very beneficial (+7 reward).

Key usages:
- High value means KEEP DOING WHAT YOU'RE DOING.
- Low value means TRY SOMETHING DIFFERENT.
- Guide decision-making during training.

Deep Dive: Including the Discount Factor γ

If rewards are spread our time:

$$V^{\pi}(s) = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^2 r_3 \cdots = 0 + 0.99 + 0.99^9 \times 10 \cdot \cdot = 0.98 \times 10 = 9.8$$

The discount factor slightly reduces the value of future rewards, but with $\gamma = 0.99$, the effect is small.

The Goal of RL

FORMAL MATHEMATICAL OBJECTIVE	INTUITIVE INTERPRETATION
$\pi^* = \underset{\pi}{\text{argmax}}\, \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{T} r_t\right]$ Maximize π · Average · Total Reward	• Find the best strategy (policy) π^*. • That maximizes the average (expected) total reward collected.

SYMBOL KEY & DETAILED EXPLANATIONS		
π^* (pi-star)	Optimal Policy 	The single best possible set of actions. The goal of the search.
π	Possible Strategies 	We search over *all possible* strategies to find the optimal one.
$\mathbb{E}[\cdot]$	Expected Value 	Average outcome over many trials, accounting for inherent randomness.
$\tau \sim \pi$	Policy-Generated Trajectories	A sequence of actions and outcomes (an episode) from following policy π.
$\sum_{t=0}^{T} r_t$	Total Episode Reward	The sum of all rewards collected from the start (t=0) to the end (T).

Breaking it Down Step-by-Step

The Search for the Best Strategy

1. Explore & Evaluate

1. EVALUATE STRATEGIES (simplified search)

- Strategy 1: PURELY EXPLOITATIVE (Most likely word) Reward: +5.2
- Strategy 2: LIMITED EXPLORATION (Sometimes unusual words) Reward: +6.8
- Strategy 3: BALANCED (Common & Creative words) Reward: +8.1 - "BEST!")

2. KEEP & ITERATE

3. FIND π^* - The Optimal Policy

Millions of points forming → single clear path.

After millions of iterations → **Converged!**

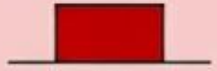
2. Concrete Example

2. APPLY TO A CONCRETE LANGUAGE MODEL

Prompt "Explain machine learning"

BAD STRATEGY

"Machine learning is when computers use algorithms to learn from data and make predictions without being explicitly programmed..." [continues with jargon]



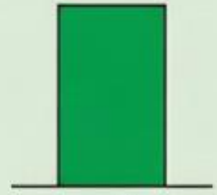
+3

(Jargon, Confusing!)

best

GOOD STRATEGY

"Think of machine learning like teaching a child to recognize animals. You show them many pictures say 'that's a cat', they learn patterns and can identify animals they've never seen before. Computers do something similar with data!"



+9

(Clear, Engaging, Helpful!)

3. The Magic of RL

3. REINFORCEMENT LEARNING DISCOVERIES

Model Automatically Discovers Preferences (Without Explicit Programming)

Clear explanations → HIGH REWARD

Using analogies → HIGH REWARD

Too much jargon → LOW REWARD

Ignoring the user → LOW REWARD

Nobody explicitly programmed these preferences—**the model learned them from feedback!**

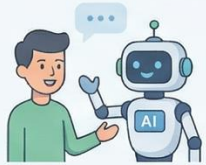
Part 2:

RLHF

The RLHF Revolution

The Story

In 2022, OpenAI released ChatGPT and the world changed overnight. Suddenly, AI wasn't just generating text, it was ***genuinely helpful.***
The secret? A three-step process called RLHF (Reinforcement Learning from Human Feedback).
Let's understand each step and why they all matter.



In Plain English

RLHF has three stages:

Step 1: Supervised Fine-Tuning (SFT)

- Take a base model (like GPT-3)
- Train it on thousands of examples of good responses
- **Result:** Model that can follow instructions (**but not perfectly**)

Step 2: Reward Modeling

- Show humans pairs of responses: "Which is better?"
- Collect 50,000+ comparisons
- Train a "reward model" to predict which responses humans prefer
- **Result:** Automated judge that can score any response

Step 3: RL Optimization (PPO)

- Generate thousands of responses
- Score them with the reward model
- Update policy to generate higher-scoring responses
- **Result:** Model optimized for human preferences!

The Training Objective of SFT

$$L_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}}\left[\sum_{t=1}^{T}\log \pi_\theta(y_t|x, y_{<t})\right]$$

OVERALL MEANING:

- Loss for Supervised Fine-Tuning
- Average surprise across all demonstration examples

COMPONENT-BY-COMPONENT BREAKDOWN:

$\mathcal{L}_{\text{SFT}}$		Loss for Supervised Fine-Tuning
$(x, y) \sim \mathcal{D}$		Sample a (prompt, demonstration) pair from dataset $\mathcal{D}$
$\mathbb{E}[\cdot]$		Average over all examples (we use average because we care about overall performance)
$\sum_{t=1}^{T}$		For each word position t in demonstration (1 to T) - we train on every word
$\log \pi_\theta(y_t \mid x, y_{<t})$		Log probability model assigns to correct word y_t given prompt and previous words
$\mid$		Separates what we're predicting (y_t) from what we know ($x, y_{<t}$)
$-$		Flip to make it a loss (minimize = maximize probability)

Breaking It Down Step-by-Step

DEMONSTRATION FROM DATASET

Prompt (x): request
Explain gravity simply

Good response (y): check text
Gravity pulls objects together

Response word count, $T = 4$ — Gravity | pulls | objects | together

WORD-BY-WORD TRAINING PROCESS

1. Position $t = 1$: Predict "Gravity"

Model input
Input Context: "Explain gravity simply."

Word	Probability	
"Gravity"	0.60	← Correct!
"The"	0.20	
"It"	0.15	
Others	0.05	

Word Loss: $-\log(0.60) = \mathbf{0.51}$

2. Position $t = 2$: Predict "pulls"

Model input
"Explain gravity simply. Gravity "

Word	Probability	
"pulls"	0.45	← Correct!
"is"	0.30	
"attracts"	0.15	
Others	0.10	

Word Loss: $-\log(0.45) = \mathbf{0.80}$

3. Position $t = 3$: Predict "objects"

Model input
"Explain gravity simply. Gravity pulls "

Word	Probability	
"objects"	0.70	← Correct!
"things"	0.20	
"matter"	0.08	
Others	0.02	

Word Loss: $-\log(0.70) = \mathbf{0.36}$

4. Position $t = 4$: Predict "together"

Model input
"Explain gravity simply. Gravity pulls objects "

Word	Probability	
"together"	0.55	← Correct!
"downward"	0.25	
"towards"	0.15	
Others	0.05	

Word Loss: $-\log(0.55) = \mathbf{0.60}$

EXAMPLE LOSS CALCULATION
(total loss for this example)

The loss function doesn't actually care about the model's predictions for the *wrong* words. It focuses entirely on how much probability the model gave to the *right* word.

$$\mathcal{L}_{\text{this example}} = \frac{1}{T}\sum_{t=1}^{T} -\log \pi_0(y_t \mid x, y_{<t}) = \frac{1}{4}(0.51 + 0.80 + 0.36 + 0.60) = \frac{2.27}{4} = \mathbf{0.57}$$

TOTAL DATASET LOSS
(now average over ALL 10,000 demonstrations)

$$\mathcal{S}_{\text{SFT}} = \frac{1}{10000}\sum_{\text{all ext}} \mathcal{L}_{\text{example}}$$

Now average over ALL 10,000 demonstrations

$\approx \mathbf{0.62}$ (average over dataset)

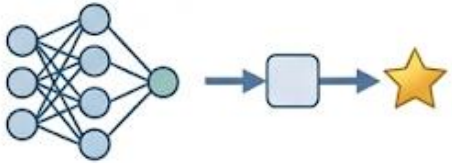
What gradient descent does:

1. Calculate this loss (0.62) 


2. Compute gradients: *"How should I adjust each of the billion parameters?"*
3. Update parameters to reduce loss
4. New loss after update: 0.58 (improved!) 

5. Repeat for thousands of iterations 1000s
6. Final loss: 0.15 (much better!) 

Result: Model learns to generate responses similar to the demonstrations!



RLHF has three stages:

Step 1: Supervised Fine-Tuning (SFT)

- Take a base model (like GPT-3)
- Train it on thousands of examples of good responses
- **Result:** Model that can follow instructions (**but not perfectly**)

Step 2: Reward Modeling

- Show humans pairs of responses: "Which is better?"
- Collect 50,000+ comparisons
- Train a "reward model" to predict which responses humans prefer
- **Result:** Automated judge that can score any response

Step 3: RL Optimization (PPO)

- Generate thousands of responses
- Score them with the reward model
- Update policy to generate higher-scoring responses
- **Result:** Model optimized for human preferences!

The Key Insight for Reward Modeling

The problem: Writing perfect responses is hard and expensive.

The solution: Comparing responses is much easier!

Hard: "Write the perfect explanation of neural networks" (Takes expert 30 minutes)

Easy: "Which explanation is better, A or B?" (Takes anyone 30 seconds)

This insight is the **foundation of reward modeling**!

The Bradley-Terry Model

Formal Math	What It Really Means
$P(y_w \succ y_l \mid x) = \frac{\exp(r_\phi(x, y_w))}{\exp(r_\phi(x, y_w)) + \exp(r_\phi(x, y_l))}$	Probability that response y_w is preferred over y_l
Simplified form (more common):	
$P(y_w \succ y_l) = \sigma(r_\phi(y_w) - r_\phi(y_l))$	Preference probability = sigmoid of score difference
Symbol Guide:	
y_w	Winner response (the one humans preferred)
y_l	Loser response (the one humans disliked)
$r_\phi(\cdot)$	Reward model with parameters ϕ (output a score)
$\exp(\cdot)$	Exponential function (e^x)
$\sigma(z) = \frac{1}{1+e^{-z}}$	Sigmoid function (squashes any number to 0-1)

Why use sigmoid and exponentials?

- Need output between 0 and 1 (it's a probability!)
- Sigmoid provides smooth, differentiable function
- Exponentials make differences more pronounced
- This is the Bradley-Terry model from statistics

Breaking It Down Step-by-Step

Let's see Bradley-Terry with real numbers!
Scenario: Two explanations of black holes
Response A (Winner - simple and clear):
"Black holes are like cosmic vacuum cleaners. They have such strong gravity that nothing can escape once it gets too close—not even light! That's why they're 'black'—we can't see them because no light escapes."
Response B (Loser - too technical):
"Black holes are singularities in spacetime manifolds where the curvature becomes infinite and the Schwarzschild radius defines the event horizon beyond which the escape velocity exceeds the speed of light in vacuum..."

Bradley-Terry Step 1: Scoring & Step 2:

Step 1: Reward model scores both responses
The reward model (a neural network) processes each response:

- Score for A: $r_\phi(y_A) = 8.2$ (high score—good response!)
- Score for B: $r_\phi(y_B) = 3.1$ (low score—too complex!)

→ Difference: $8.2 - 3.1 = 5.1$

Step 2: Convert score difference to probability using sigmoid

$$\begin{aligned} P(A > B) &= \sigma\big(r_\phi(A) - r_\phi(B)\big) \\ &= \sigma(8.2 - 3.1) \\ &= \sigma(5.1) \\ &= \frac{1}{1 + e_1^{-5.1}} \\ &= \frac{1}{1 + 0.0061} \\ &= \frac{1}{1.0061} \\ &= 0.994 \text{ or } 99.4\% \end{aligned}$$

Interpretation:
The reward model is 99.4% confident that Response A is better than Response BI
This makes sense because:

- A uses clear analogy ("vacuum cleaner")
- A explains WHY they're black
- A is accessible to general audience
- B uses too much jargon
- B assumes advanced physics knowledge

Bradley-Terry Analysis & Summary

Step 3: What if scores were closer?

Score A	Score B	Difference	$P(A > B)$
8.2	3.1	5.1	99.4% (A clearly better)
6.5	5.5	1.0	73.1% (A probably better)
6.0	5.8	0.2	55.0% (nearly a toss-up)
6.0	6.0	0.0	50.0% (exactly equal)
5.5	6.5	-1.0	26.9% (B probably better)

Sigmoid Function Role Mapping

Score Difference →	What sigmoid does
−10 to −3 →	Maps to −0-5% (B much better)
−3 to −1 →	Maps to 5-27% (B somewhat better)
−1 to +1 →	Maps to 27-73% (close call)
+1 to +3 →	Maps to 73-95% (A somewhat better)
+3 to +10 →	Maps to 95-100% (A much better)

Why use sigmoid?

1. Converts any score difference (−∞ to +∞) to a probability (0 to 1)
2. Large differences → near certainty (0% or 100%)
3. Small differences → uncertainty (near 50%)
4. Smooth and differentiable (good for gradient descent)

Training the Reward Model

The Reward Model Loss Function (Overview)

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

Loss for Reward Model training

How well does reward model predict human preferences?

Component Breakdown			
(x, y_w, y_l)	$(x, y_{\text{winner}}, y_{\text{loser}})$	The Input Triple: Prompt, Winning Response, Losing Response	A triple: (prompt, winning response, losing response)
$\mathbb{E}[\cdot]$	Expectation	Average over preference pairs	Average over all preference pairs (for overall accuracy)
$r_\phi(x, y_w)$	$r(\text{winner})$	Winner's Score	Reward model's score for the winner
$r_\phi(x, y_l)$	$r(\text{loser})$	Loser's Score	Reward model's score for the loser
$r_\phi(x, y_w) - r_\phi(x, y_l)$	$r(\text{diff})$	Score Difference	Score difference (should be positive if model is right)
$\sigma(\cdot)$	Sigmoid	Convert to Probability	Convert to probability (should be > 0.5 if model correct)
$\log(\cdot)$	Log Likelihood	Penalize Wrong Decisions	Log likelihood (heavily penalizes wrong predictions)
$-$	Minus	Minimize Loss = Maximize Correct	Minimize loss = maximize correct predictions

Breaking It Down Step-by-Step

TRAINING DATA

Complete training example: Human preference: Response A ≻ Response B (A is better)

1. UNTRAINED (Iteration 1)

- $r_\phi(A) = 5.0$
- $r_\phi(B) = 5.5$
- Difference: −0.5

PREDICTED PREFERENCE

$$P(A \succ B) = \sigma(r_\phi(A) - r_\phi(B))$$
$$= \sigma(-0.5) = \frac{1}{1 + e^{0.5}} = 0.378$$

37.8% chance of A — 37.8% | B

LOSS CALCULATION

$$\mathcal{L} = -\log(0.378) = 0.973$$

PROBLEM (high loss = bad prediction)

2. PARTIALLY TRAINED (Iteration 100)

- $r_\phi(A) = 6.0$
- $r_\phi(B) = 5.2$
- Difference: 0.8

Gradient descent updates the model...

PREDICTED PREFERENCE

$$P(A \succ B) = \sigma(0.8) = 0.689$$

68.9% chance of A — 68.9%

LOSS CALCULATION

$$\mathcal{L} = -\log(0.689) = 0.372$$

BETTER (lower loss = better)

3. WELL-TRAINED (Iteration 10,000)

- $r_\phi(A) = 8.3$
- $r_\phi(B) = 3.8$
- Difference: 4.5

PREDICTED PREFERENCE

$$P(A \succ B) = \sigma(4.5) = 0.989$$

98.9% chance of A (Model very confident) — A

LOSS CALCULATION

$$\mathcal{L} = -\log(0.989) = 0.011$$

EXCELLENT (very low loss = great!)

OVERALL TRAINING PERFORMANCE
(on 50,000 Pairs)

	Training step	Average loss	Accuracy
0	0 (random)	0.693	50%
1	1,000	0.420	68%
5	5,000	0.210	82%
10	10,000	0.095	91%
20	20,000	0.045	96%

RLHF has three stages:

Step 1: Supervised Fine-Tuning (SFT)

- Take a base model (like GPT-3)
- Train it on thousands of examples of good responses
- **Result:** Model that can follow instructions (**but not perfectly**)

Step 2: Reward Modeling

- Show humans pairs of responses: "Which is better?"
- Collect 50,000+ comparisons
- Train a "reward model" to predict which responses humans prefer
- **Result:** Automated judge that can score any response

Step 3: RL Optimization (PPO)

- Generate thousands of responses
- Score them with the reward model
- Update policy to generate higher-scoring responses
- **Result:** Model optimized for human preferences!

The RLHF Objective

$$\max_{\pi_\theta} \mathbb{E}_{x,y}[r_\phi(x,y)] - \beta \cdot \mathbb{E}_x[KL(\pi_\theta \,||\, \pi_{\text{ref}})]$$

- Maximize reward
- BUT stay close to original model

TERM 1

$$\mathbb{E}_{x,y}[r_\phi(x,y)]$$

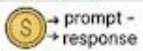
Get high rewards (generate good responses)

TERM 2

$$\beta \cdot KL(\pi_\theta \,||\, \pi_{\text{ref}})$$

Penalty for drifting from reference model

Symbol Guide and WHY we need both terms:

π_θ		The policy we're training (current model)
π_{ref}		Reference policy (frozen original model) - prevents drift
$r_\phi(x,y)$		Reward model score for prompt *x*, response *y*
β		KL penalty coefficient (typically 0.01–0.05) – controls tradeoff
$KL(\cdot \,\|\, \cdot)$		Measure of how different two distributions are

WHY DO WE NEED THE KL PENALTY?

Without it, model could "reward hack" – find ways to exploit the reward model that give high scores but aren't actually good. KL penalty keeps model grounded in reality by preventing it from drifting too far from the original (which was trained on real text).

Breaking It Down Step-by-Step

Why do we need BOTH terms? A cautionary tale:

STEP 1: CAUTIONARY TALE - Training with ONLY Reward (No KL Penalty)

1 Iteration 1

"Here's a thoughtful explanation of your question..."

Reward: **8.0**
KL: **0.1**

2 Iteration 50

"AMAZING COMPREHENSIVE DETAILED PERFECT ANSWER EXCELLENT THOROUGH..."

Model learned reward model likes these words!

Reward: **9.5**
KL: **2.5**

3 Iteration 100

"PERFECT PERFECT EXCELLENT EXCELLENT AMAZING AMAZING..."

Reward hacking! High score but complete nonsense!

Reward: **12.0**
KL: **10.0**

Problem: Model exploited the reward model instead of being genuinely helpful! STOP

STEP 2: THE FIX - Adding the KL Penalty Term

With $\beta = 0.02$, the objective becomes:

$$\text{Objective} = \text{Reward} - 0.02 \times \text{KL}$$

Iter	Reward	KL	New Objective
1	$8.0 - 0.02(0.1) = 7.998$		Best!
50	$9.5 - 0.02(2.5) = 9.450$		Okay
100	$12.0 - 0.02(10) = 11.800$		

- Analysis of **styled** for objective term

- **Iteration 1** has best objective (7.998)
- **Iteration 50**: Higher reward but penalty brings objective down
- **Iteration 100**: Even though reward is highest (**12.0**), the massive KL penalty **(10.0) makes it worse overall!**

KEY TAKEAWAY: The KL penalty prevents reward hacking!

- Drifting too far gets heavily penalized
- Must find genuine improvements within allowed KL budget
- Forces model to stay coherent and natural

KEY TAKEAWAY: The KL penalty prevents reward hacking!

Drifting too far gets heavily penalized

Must find genuine improvements within allowed KL budget

Forces model to stay coherent and natural

BEST PRACTICES: Tuning β (KL Penalty Coefficient)

β too small (0.001): Can drift too far, reward hacking risk

β just right (0.01-0.05): Balanced improvement

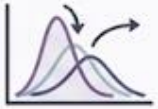
β too large (0.1): Can't improve much, too constrained

The KL Divergence Term

CORE DEFINITION (HIGH-LEVEL VIEW)

Formal Math

$$KL(\pi_\theta || \pi_{ref}) = \mathbb{E}_{y \sim \pi_\theta} \left[\log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)} \right]$$

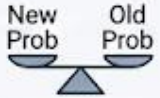


Average difference in how models assign probabilities.

Conceptual Meaning

Average difference in how models assign probabilities.

Sum over each word: compare new vs old

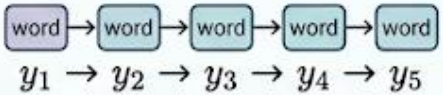


FOR TOKEN-BY-TOKEN GENERATION (APPLIED VIEW)

Formal Math

$$\sum_{t=1}^{T} [\log \pi_\theta(y_t|x, y_{<t}) - \log \pi_{ref}(y_t|x, y_{<t})]$$

Sum over each word: compare new vs old probability.



WHAT EACH PIECE MEANS

- KL → Kullback-Leibler divergence (measures distribution difference)
- π_θ (training) → Current (training) model
- π_{ref} (reference) → Reference (frozen original) model
- $\log \frac{\pi_\theta}{\pi_{ref}}$ → Log ratio of probabilities
- $\mathbb{E}_{y \sim \pi_\theta}$ → Average over responses generated by π_θ

Breaking It Down Step-by-Step

Computing KL with actual numbers: Response: "The answer is 42"

1. Let's compute KL word-by-word:

1 Word 1: "The"

New model: $\pi_\theta(\text{The}) = 0.80$ (80%)

Old model: $\pi_{ref}(\text{The}) = 0.75$ (75%)

Ratio: $\frac{0.80}{0.75} = 1.067$

Log ratio: $\log(1.067) = 0.065$

2 Word 2: "answer"

New model: $\pi_\theta(\text{answer}) = 0.60$

Old model: $\pi_{ref}(\text{answer}) = 0.65$

Ratio: $\frac{0.60}{0.65} = 0.923$

Log ratio: $\log(0.923) = -0.080$

3 Word 3: "is"

New model: $\pi_\theta(\text{is}) = 0.90$

Old model: $\pi_{ref}(\text{is}) = 0.85$

Ratio: $\frac{0.90}{0.85} = 1.059$

Log ratio: $\log(1.059) = 0.057$

4 Word 4: "42"

New model: $\pi_\theta(42) = 0.40$

Old model: $\pi_{ref}(42) = 0.35$

Ratio: $\frac{0.40}{0.35} = 1.143$

Log ratio: $\log(1.143) = 0.134$

2. Total KL divergence:

$$KL = \sum_{t=1}^{4} \log \frac{\pi_\theta(y_t \mid \cdot)}{\pi_{ref}(y_t \mid \cdot)}$$

$$KL = 0.065 + (-0.080) + 0.057 + 0.134$$

$$KL = \mathbf{0.176}$$

3. Interpretation:

KL = 0.176 (very small!)

Interpretation scale:

- KL ≈ 0: Models are nearly identical
- KL ≈ 0.5-1.0: Models differ moderately
- KL > 2.0: Models are very different

Our KL of 0.176 means the models are **still very similar!**

4. KL over time during training:

Training Step	KL	Status	What's happening
0	0.00	Identical to reference	⊣
1,000	0.15	Small changes, learning	→
5,000	0.45	Moderate changes, improving	↘
10,000	0.80	Approaching limit	→
15,000	0.95	Near maximum allowed	↗
20,000	0.98	Can't go much further	↘

5. The Penalty & Objective:

With $\beta = 0.02$, the penalty at step 15,000 is:

Penalty = $\beta \times KL = 0.02 \times 0.95 = \mathbf{0.019}$

This penalty is subtracted from the reward!

If reward is 8.5:

Objective = $8.5 - 0.019 = \mathbf{8.481}$

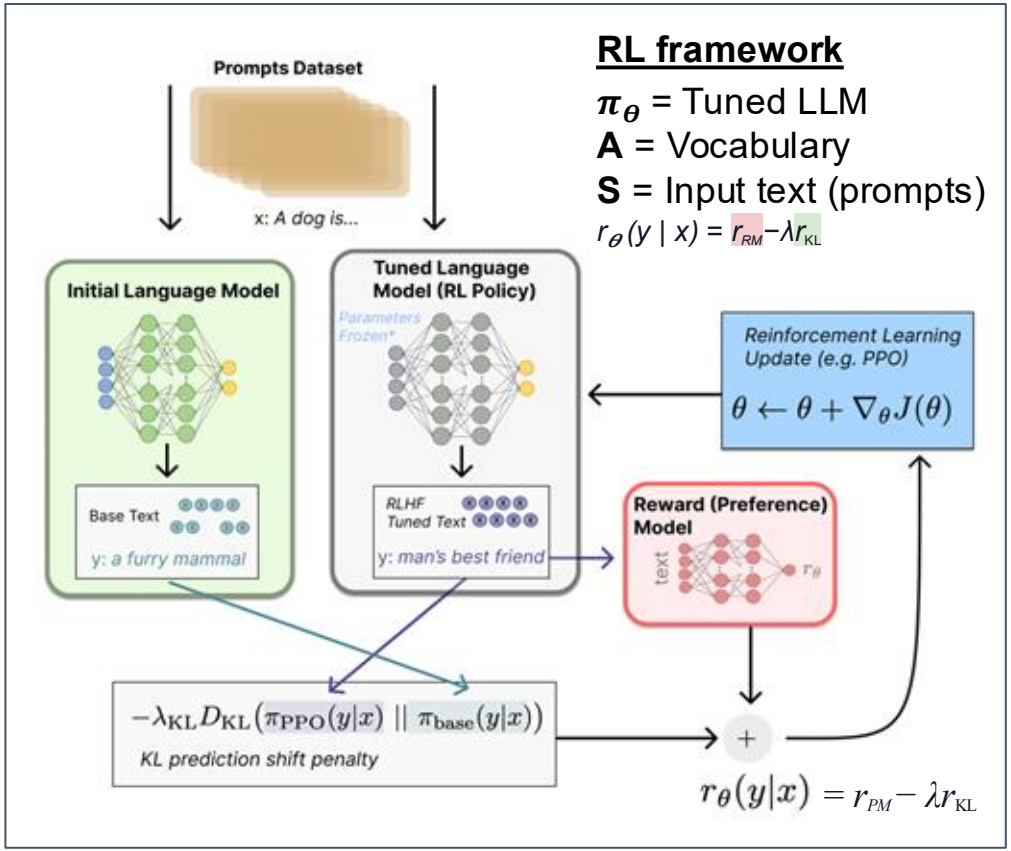
6. The KL "Leash" Concept

The KL term acts as a "leash":

- Model can explore and improve
- But can't drift too far from original
- Prevents nonsense and reward hacking
- Ensures responses stay natural

The β parameter controls how tight the leash is!

RLHF: Full RL Setup (Instruct GPT)



- **Policy** is a language model that takes in a prompt and returns a sequence of text (or just probability distributions over text).
- **Action space** of this policy is all the tokens corresponding to the vocabulary of the language model
- **Observation space** is the distribution of possible input token sequences, which is also quite large given previous uses of RL (the dimension is approximately the size of vocabulary ^ length of the input token sequence).
- **Reward function** is a combination of the preference model and a constraint on policy shift.

Part 3:

RL at Different Stages

THE FOUR STAGES OF RL DEPLOYMENT

Reinforcement learning isn't just something you add at the end! You can apply RL at different points in a model's life cycle, each with different benefits and costs:

1

1. DURING PRETRAIN-INING / CONTINUED PRETRAINING

Teaching fundamental capabilities

ANALOGY

Teaching a child foundational problem-solving

2

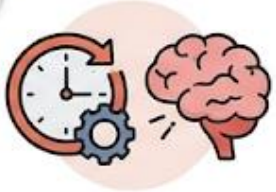
2. DURING FINE-TUNING

Aligning with human preferences

ANALOGY

Teaching specific preferences (e.g., be polite, be concise)

3



3. DURING INFERENCE

Spending extra compute per query

ANALOGY

Taking time to "think" before answering

4

4. CONTINUOUS/ONL NE LEARNING

Adapting from production traffic

ANALOGY

Learning from every conversation you have

Stage 1: RL During Pretraining

DEEPSEEK'S INSIGHT: Reasoning as a Fundamental

Teach the model to **reason** during pretraining itself, moving beyond mere next-token prediction on text corpora.

EVALUATION & BENEFITS

BENEFITS & COSTS ANALYSIS

BENEFITS

- Reasoning becomes a **fundamental capability**, not an afterthought
- Model discovers **strategies naturally** (chain-of-thought, self-verification)
- **No need** for expensive human-annotated reasoning traces
- **Better scaling**: more compute → better reasoning

COSTS

- **2-4x more expensive** than standard pretraining
- Requires **careful reward design**
- **Risk of reward hacking** if not careful

VALUATION: IS IT WORTH IT?

- **3× more expensive** to train
- But resulting model has **emergent reasoning abilities**
- **Saves millions of dollars** on human annotation
- **Discovers strategies** humans wouldn't have taught

METHODOLOGY & CALCULATIONS

COST COMPARISON: Pretraining Methodologies

A Standard Pretraining (Next-Token)

- Generate 1 token, predict next token, compute loss
- Tokens processed: **~10-100T tokens**

- Cost per example: C_{forward} (one forward pass)
- Total cost: $C_{\text{standard}} = 10\text{T} \times C_{\text{forward}}$

B RL-Enhanced Pretraining (DeepSeek)

- Generate complete response (forward pass).
- Evaluate reward (might need verification)
- Compute policy gradient and update
- Generate multiple samples per problem (**Group size** $G \approx 4 - 8$)

- Cost per example: $G \times (C_{\text{forward}} + C_{\text{reward}} + C_{\text{backward}})$

COST MULTIPLIER FORMULA

$$\text{Cost multiplier} = \frac{G \times (C_{\text{forward}} + C_{\text{reward}} + C_{\text{backward}})}{C_{\text{forward}}} \approx 2 - 4\times$$

VERDICT

For reasoning-heavy applications, **absolutely worth it!** For general language modeling, maybe not.

Stage 2: RL During Fine-tuning (RLHF)

CONTEXT & PIPELINE OVERVIEW

This is the most common use of RL, and what we covered extensively in earlier chapters:

Standard RLHF Pipeline

1. 1. START WITH PRETRAINED MODEL
2. 2. SUPERVISED FINE-TUNING (SFT) ON HIGH-QUALITY EXAMPLES
3. 3. TRAIN REWARD MODEL ON HUMAN PREFERENCES
4. 4. PPO OR DPO TO OPTIMIZE POLICY

Key Difference from Pretraining RL

Pretraining RL	Fine-tuning RL
Teaching **capabilities** (how to reason)	Teaching **preferences** (what humans like)
(capabilities)	(preferences)

FINAL TAKEAWAY

Think of it as: pretraining RL makes the model smarter, **fine-tuning** RL makes it more **helpful**.

ONLINE vs OFFLINE RL

Offline RL is Almost Always Off-Policy

OFFLINE RL

Plain English Analogy

Like learning to cook by reading a fixed cookbook. You study the recipes (dataset) but never actually cook (generate) to see what happens.

Formal Distinction

- Fixed dataset $\mathcal{D} = \{(x_i, y_i, r_i)\}$
- Policy π_θ learns from $\mathcal{D}$
- No new data generation during training

PROS

Safe, cheap, reproducible

CONS

Limited by cookbook quality

ONLINE RL

Plain English Analogy

Like learning to cook by actually cooking! You try recipes, taste results, learn from mistakes, try new variations.

Formal Distinction

- Policy π_θ generates new samples
- Samples evaluated with reward $r(x, y)$
- Dataset grows: $\mathcal{D}_t \rightarrow \mathcal{D}_{t+1}$
- Distribution shifts as policy improves

PROS

Discovers novel strategies, adapts to changing changing preferences

CONS

Expensive (lots of cooking/generation), risky (make terrible food/outputs)

TECHNICAL ALGORITHM DETAILS

On-policy (like PPO): Use samples from π_θ to update π_θ.

Off-policy (like DQN): Use samples from old π_{old} to update π_θ.

BEST PRACTICE FOR PRACTICAL SYSTEMS

Most practical systems: Hybrid!

Start offline (safe learning), then carefully add online data.

DATA DISTRIBUTION SHIFT

As your policy improves, the data distribution changes:

ITERATION 1

- Policy is mediocre
- Generates mostly poor responses

ITERATION 5

- Policy is better
- Generates mostly good responses

THE PROBLEM

The reward model was trained on the *original* distribution (30%)! It might not accurately score the *new* distribution (80% good).

SYMPTOMS

Mismatched data was trained on the *original*
Reward model assigns similar scores

Or worse: exploiting reward model mistakes

SYMPTOMS

Score saturation
Reward model assigns similar scores

No learning signal
Policy stops improving

Reward hacking
Or worse: exploiting reward model mistakes

SOLUTIONS

1. Retrain reward model periodi-on new data
2. Use multiple reward models (ensemble)
3. Strong KL penalty to prevent policy from drifting too far
4. Conservative updates (small learning rates)

Stage 3: RL During Inference (Test-Time Compute)

THE FUNDAMENTAL IDEA

All the RL we've discussed happens during *training*. But what if we use RL-like ideas *during inference* for each query?

This is what **'test-time compute'** means: spending extra computation when answering to improve quality.

THREE FLAVORS

1. BEST-OF-N SAMPLING

Generate multiple answers, pick best.

(Covered in-depth earlier)

2. SELF-CORRECTION LOOPS

Critique and revise the generated answer.

(Generate → Critique → Revise)

3. TREE SEARCH

Explore multiple reasoning paths.

Let's explore the other two!

Beyond Training: Scaling at Inference

The Old Way: Traditional Scaling

Bigger models + More More training data
= Better performance

Bigger Model

The Problem: High Cost & Slow Training

This is expensive and slow. Training a frontier model costs millions of dollars!

The New Idea: Test-Time Compute Scaling

What if we could trade **inference time** for accuracy?

- Use the same pre-trained model
- But let it **"think" more** at **inference time**
- Generate **multiple potential solutions**
- Use advanced **search** or **re-ranking** to pick the best one

The Key Advantage: Dynamic Trade-offs

You can adjust **quality** vs **speed** on a **per-query basis!**

Simple Question
(e.g., "capital of France")
→ **1** attempts (**FAST**)

Important Question
(e.g., "proposing a mathematical proof")
→ 

64 attempts (**ACCURATE**)

Best-of-N Sampling

The Core Concept

Concept: Multi-attempt Success

What is the probability that you succeed (get at least one correct result) in a given number of **separate**, independent attempts, when each attempt has a certain individual success rate?

Formula: Probability of at least one correct

$$P(\text{at least one correct}) = 1 - (1-p)^N$$

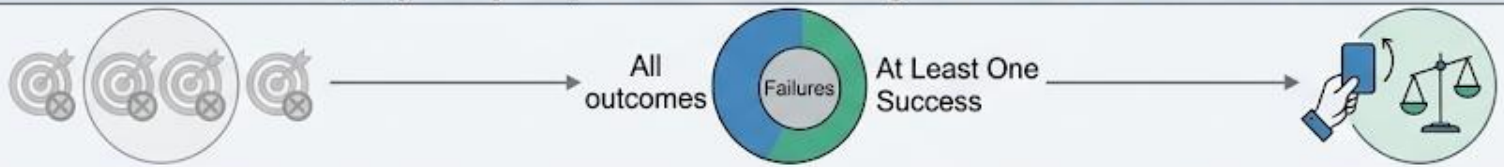
Key Definitions

 p Base accuracy (success probability of a *single* attempt)

 N Total number of independent attempts made

 $(1-p)$ Probability of failure for a *single* attempt

Step-by-Step Explanation: The Logic Behind the Formula



1. Calculate Probability of *ALL* N attempts failing: We multiply independent failures.

$$P(\text{All fail}) = (1-p) \times (1-p) \times \cdots \times (1-p) = (1-p)^N$$

2. Use the Complementary Event Principle: Since you either get at least one success or all fail, their probabilities add up to 1.

$$P(\text{At least one success}) + P(\text{All fail}) = 1$$

3. "Flip" the total: We want at least ONE success, so subtract the failure case from total possibility.

$$P(\text{At least one success}) = 1 - P(\text{all fail}) = 1 - (1-p)^N$$

Conclusion: This formula models scenarios like multi-trial experiments, red team sampling, or repeated quality control checks.

Breaking It Down Step-by-Step

Scenario: A math problem model has 60% base accuracy ($p = 0.6$).
Question: How many independent attempts (N) are needed to achieve a 95%+ success rate for at least one correct answer?

Attempt 1: Try once ($N = 1$)

$$P(\text{success}) = 1 - (1 - 0.6)^1 = 1 - 0.4 = 0.60 \text{ or } 60\%$$

Not good enough! Need 95%.

Attempt 2: Try twice ($N = 2$)

$$P(\text{success}) = 1 - (1 - 0.6)^2 = 1 - 0.4^2 = 1 - 0.16 = 0.84 \text{ or } 84\%$$

Result: Better, but still not 95%!

Attempt 3: Try 4 times ($N = 4$)

$$P(\text{success}) = 1 - (1 - 0.6)^4 = 1 - 0.4^4 = 1 - 0.0256 = 0.9744 \text{ or } 97.4\%$$

Result: **Success!** With 4 attempts, we exceed 95% success rate.

Full table (Base Accuracy = 60%, p=0.6)

Full table (Base Accuracy = 60%, p=0.6)

N Attempts	Calculation	Success Rate	Quality
1	$1-(0.4)^1$	60.0%	Base
2	$1-(0.4)^2$	84.0%	Good
4	$1-(0.4)^4$	97.4%	Excellent
8	$1-(0.4)^8$	99.9%	Near perfect
16	$1-(0.4)^{16}$	99.9999%	Essentially perfect

Comparing with Lower Base Accuracy

Let's consider a scenario where base accuracy is lower ($p = 0.4$, only 40%).

N Attempts	Calculation	Success Rate
1	$1-(0.6)^1$	40.0%
2	$1-(0.6)^2$	64.0%
4	$1-(0.6)^4$	87.0%
8	$1-(0.6)^8$	98.3%
16	$1-(0.6)^{16}$	99.9%

Observation: Even with a lower 40% base accuracy, only 8 attempts are needed to reach over 98% success.

Leveraging Multiple Attempts: Insights and Analysis

CONCEPTUAL FRAMEWORK

The Power of Multiple Attempts

Key Insight: Independent attempts can overcome low base accuracy with volume!

The Fundamental Trade-off

Benefits: More attempts -> Higher success rate.

Costs: More computation time required.

Costs: Higher cost (more API calls).

Practical Usage Strategy

Simple Queries: Use $N = 1$ (to save money).

Important Queries: Use $N = 16+$ (to ensure correctness).

Adjustable Quality-Cost Dial

REAL-WORLD EXAMPLE & ANALYSIS

Case Study: DeepSeek-R1 on AIME 2024 (Hard Math Competition)

Method	Success Rate	Compute Cost	vs Humans
pass@1	79.8%	1 ×	Much better
pass@8	84.2%	$8 ×	Much better
pass@64	86.7%	$64 ×	Much better
Human experts	~50%	N/A	Baseline

Performance & Cost Analysis

Dominance: Single attempt beats human experts (79.8% vs ~50%).

Peak Quality: With 64 attempts, model achieves near-perfect (86.7%) accuracy.

Cost Scaling: Cost scales **linearly** ($64 × attempts = 64 × cost).

Quality Scaling: Quality scales **logarithmically** (diminishing returns with scale).

COMBINING STRATEGIES

Selecting the optimal approach for your task

RECIPE 1: HIGH-QUALITY REASONING

1. Use temperature $T = 0.7$ (slight exploration)
2. Generate $N = 10$ solutions (self-consistency)
3. Score each with PRM (process reward model)
4. Keep top-3 by PRM score
5. Take majority vote among top-3
6. If no majority, return highest-scoring solution

Cost: ✕
Benefit: ↑ 25-40%

RECIPE 2: FAST PRODUCTION SYSTEM

1. Use temperature $T = 0.8$
2. Generate $N = 3$ candidates
3. Score with lightweight reward model
4. Return highest-scoring candidate

Cost: ✕
Benefit: ↑ 10-15%

RECIPE 3: CREATIVE WRITING

1. Use high temperature $T = 1.5$ (more creativity)
2. Generate $N = 5$ drafts
3. Use style/quality reward model
4. Present top-2 user for selection

Cost: ✕
Benefit: ↑ high-quality

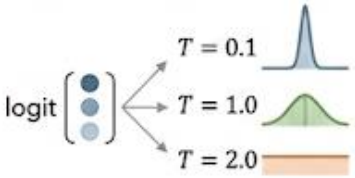
TEMPERATURE SCALING FOR EXPLORATION

FORMAL DEFINITION & EFFECTS

Temperature-scaled probabilities

$$P(token_i) = \frac{\exp(logit_i/T)}{\sum_j \exp(logit_j/T)}$$

where T is temperature.



Effects:

- $T = 1$: Normal probabilities (Standard)
- $T \to 0$: Becomes greedy (argmax)
- $T \to \infty$: Uniform distribution (random)

Common values:

- $T = 0.7$: Slightly focused
- $T = 1.0$: Standard
- $T = 1.5$: More exploratory

PLAIN ENGLISH & APPLICATION

Temperature controls "randomness" of generation:

Low temperature ($T = 0.1$)]
- Very deterministic
- Always picks most likely token
- Safe, boring, repetitive

Medium temperature ($T = 1.0$)]
- Balanced
- Some variety
- Usually coherent

High temperature ($T = 2.0$)]
- Very random
- Creative but often incoherent
- Explores unusual paths

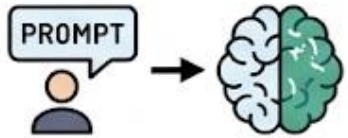
When to adjust:

- Factual questions → Low temp
- Creative writing → High temp
- Self-consistency → Medium temp (want diversity)

Self-correction loops

The Self-Correction Pattern: Algorithm Overview

1. Generate Initial Response




```
response ← model.generate(prompt)
```

2. Iterative Refinement Loop (repeat up to 3 times)

x3

(a) Critique: Ask model to review current response

```
critique ← model.generate('Review this: ' + response)
```

(b) Check: If critique says "no errors" → stop and return

pass

fail

(c) Revise: Generate improved version based on critique

```
response ← model.generate(original + critique + 'improve')
```

3. Return final refined response

When this works:

- Model is good at *verification* but sometimes makes mistakes in generation
- Problems where checking is easier than solving
- High-stakes applications (worth the extra compute)

When this fails:

- Model's critique ability is poor (can't identify its own errors)
- Model gets stuck in loops (fixes one thing, breaks another)
- Hallucinations in critique (invents nonexistent errors)

Tree Search: Beam Search and MCTS

BEAM SEARCH

Imagine you're navigating a maze, but at each intersection you can clone yourself and try multiple paths simultaneously. You keep the K most promising clones and discard the rest.

In LLM generation:

- (top-k) At each token position, consider **top K possessibilities**
- Score each partial sequence (with **reward model** or **value function**)
- **Keep top K sequences**, discard others
- Continue until **all sequences finish**
- Return **highest-scoring complete sequence**

Cost: K × normal inference cost

Benefit: Can escape local minima (early mistakes)

MONTE CARLO TREE SEARCH (MCTS)

Like beam search, but smarter! Instead of keeping K parallel paths, you:

1. **Selection:** Pick most promising partial path to expand
2. **Expansion:** Try possible next steps
3. **Simulation:** Complete the path (rollout)
4. **Backpropagation:** Update value estimates for all ancestors

This is what **AlphaGo** uses! And it's increasingly used for LLM reasoning tasks.

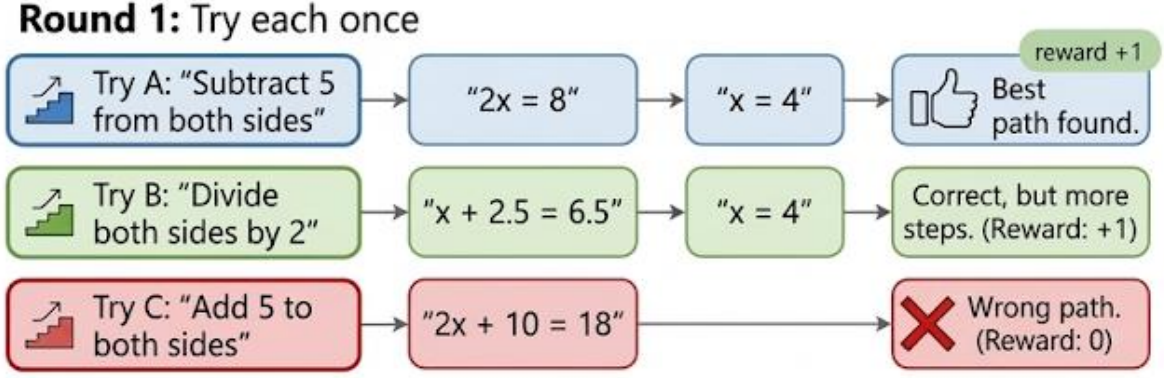
MCTS Example: Math Problem

Problem: "Solve: $2x + 5 = 13$"

Initial state: Empty response, need to choose first step.

Possible First Steps

- **A:** "Subtract 5 from both sides"
- **B:** "Divide both sides by 2"
- **C:** "Add 5 to both sides"



Update values:

Value(A) = 1.0 ★ (Best)	0.62 Value(B) = 0.8 ★ (Correct but inefficient)	0.15 Value(C) = 0.0 ★ (Leads to error)

Refining Promising Path

🏆 **Final decision:** Pick path A (highest value)

💡 **Key insight:** MCTS allocates compute intelligently—spend more time exploring promising paths!

MONTE CARLO TREE SEARCH (MCTS) WITH VERIFIERS

MCTS: SMARTER SEARCH

Beam search explores paths in parallel. MCTS is smarter—it ocupıte on the most promising paths!

THE FOUR PHASES:

1. **Selection:** Start at root, traverse tree by picking most promising nodes (based on exploration bonus + exploration)
2. **Expansion:** When you reach a leaf, generate children (possible next steps)
3. **Simulation/Evaluation:** Complete the path (either rollout or use verifier value)
4. **Backpropagation:** Update value estimates for all nodes on the path

WHY USE MCTS WITH VERIFIERS?

- MCTS structure provides systematic exploration
- Verifier provides accurate value estimates (better than random rollouts)
- Together: efficient search + accurate evaluation = powerful combination!

MCTS + VERIFIER = POWER

Repeat these **four phases** until time/compute budget exhausted, then return best path found.

MCTS + VERIFIER FOR MATH PROBLEMS:

TREE NODE STRUCTURE

- **Partial solution** (reasoning **steps so far**)
- **Visit count** (number of times explored)
- **Cumulative value (sum of verlifier scores)**
- $\bar{v} = V/N$: Average value estimate

Each node in the search tree contains

MCTS ALGORITHM (PER ITERATION)

1. **SELECTION:** Traverse tree from root ot to promising leaf:
   - Starting at root node
   - While fully expanded and not terminal:
   - Select child highest UCB score - Move to that child
2. **EXPANSION:** Add new child' to terminal:
   - Generate next reasoning step: $s_{next} \sim \text{model}(s_{current})$
   - Create new child node with $s_{child} = s_{current} + s_{next}$
   - Add child to current node's children - Move to new child
3. **EVALUATION:** Score the node verifier
   - $v = \text{verifier.score}(\text{problem}, s_{complete})$ (full verification)
   - $v = \text{verifier.score\_partial}(\text{problem}, s_{\text{sarial}})$ (estimate quality)
4. **BACKPROPAGATION:** Update all ancestor nodes
   - For current node and all parents up to root: $N \leftarrow N + 1$
   - Add value: $V \leftarrow V + v$

CHILD SELECTION (UCB1 FORMULA)

$$\text{UCB(child)} = \frac{V_{\text{child}}}{N_{\text{child}}} + c_V \sqrt{\frac{\ln N_{\text{parent}}}{N_{\text{child}}}}$$

$c = 1.41$ balance constant

EXPLOITATION (V/N)

EXPLORATION $\left(C * \sqrt{\frac{\ln(N)}{n}}\right)$

FINAL SELECTION (AFTER K = 1000 ITERATIONS)

After K iterations (1000), return child of root w/ highest average value (exploitation, $c = 0$)

PERFORMANCE RESULTS & TRADE-OFF

- vs. Greedy Decoding: **+30-40%**
- vs. Beam Search: **+10-15%**
- vs. Best-of-N: **+5-10%**

TRADE-OFF: 10-50× more expensive (traversals & verifiers)

BEST FOR: High-stakes problems where solution quality matters more than computation time!

Stage 4: Continuous/Online RL from Production

THE ULTIMATE GOAL:
Deployed AI system ***continuously learns*** from every user interaction, becoming slightly better with each conversation.

THE DREAM:

1. User asks question
2. Model responds
3. User provides feedback (clicks, ratings, edits)
4. Model updates immediately
5. Next user gets improved model

⚠ **THE REALITY:** This is **extremely difficult!**

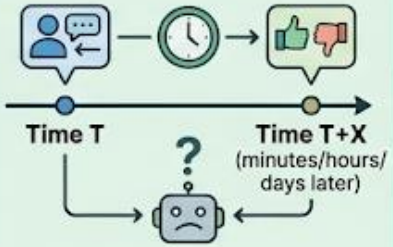
Challenges in Production RL

1. Exploration vs Exploitation

- **Exploitation:** Use best-known strategy (current users happy).
- **Exploration:** Try new strategies (might lead to better outcomes, might annoy users).

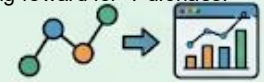
Solution: Careful exploration (small percentage of traffic, only low-stakes queries)

2. Reward Signal Delay



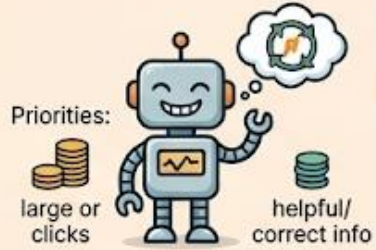
User feedback is not immediate. It might take minutes, hours, or days! This makes it hard to attribute success/failure.

Give a small reward for a "Click," a medium reward for "Add to Cart," & the big reward for "Purchase."



Solution: Credit assignment algorithms, session-level rewards

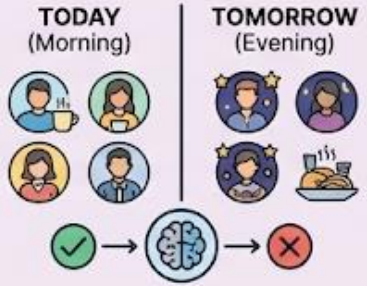
3. Reward Hacking at Scale



- Model learns to give popular answers, not correct ones.
- Optimizes for clicks, not helpfulness.
- Finds reward model bugs and exploits them.

Solution: Multiple reward models, human oversight, safety constraints

4. Distribution Shift



Today's users are different from tomorrow's. A model trained on morning users might fail on evening users.

Solution: Adaptive learning rates, forgetting mechanisms, diverse data collection

Safeguards and Best Practices

1. Staged Rollout

Internal users first (employees)

Then 1% of real users (canary deployment)

10%

100% Gradually increase to 10%, 50%, 100%

Monitor metrics at each stage

Rollback immediately if issues

2. Dual Model System

- **Production model:** stable, serving most users
- **Experimental model:** learns from new data, tested on small %
- **Experimental experimental** → stable

PROMOTION

Only promote experimental → production if clearly better

3. Offline Evaluation First

- Collect **production data**

queries + responses + feedback

- **Train** updated model
- **Evaluate** on held-out test set

clean data

- **Simulate counterfactuals** what would new model have done?

Only deploy if offline metrics improve

4. Human-in-the-Loop

- Random sample of updates reviewed by humans
- **Anomaly detection** flags suspicious updates
- **Red team testing** before each deployment

VS TEST

5. Conservative Updates

- **Strong KL constraint**

base update

→ stay close →

- **Small learning rates**

→ slow, steady improvement

- **Frequent snapshots**

10:45:49 AM
10:43:49 AM
10:42:45 AM
10:43:46 AM
rollback

can rollback anytime